Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251214**
Nama Lengkap	**Raffiel Gideon Nyolo Nyolo**
Minggu ke / Materi	**09 / Pengolahan String dan Regular Expression**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

A. Pengantar String

String adalah kumpulan karakter yang membentuk satu kesatuan untuk menyimpan teks dalam program, baik panjang maupun pendek, dan biasanya direpresentasikan dalam kode ASCII. String bukan tipe data dasar karena menyimpan lebih dari satu nilai dalam satu variabel, dan dalam beberapa bahasa pemrograman (seperti C) tidak memiliki tipe khusus, melainkan diperlakukan sebagai array atau kumpulan karakter.

B. Pengaksesan String dan Manipulasi String

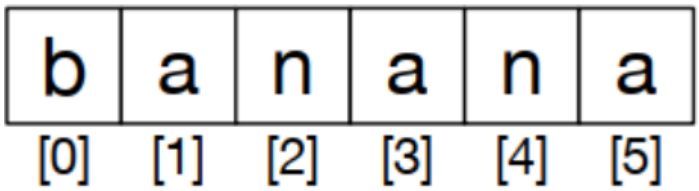
String dapat dibuat dengan sederhana menggunakan variabel:

```
namasaya = "Raffiel Gideon"
temansaya1 = "Jingjong"

print(temansaya1)
print(namasaya[5]) #"i"

huruf = temansaya1[0]
print(huruf) #'R'
```

String dibuat dengan deklarasi variabel yang langsung berisi data, dan dapat diakses sebagai satu kesatuan melalui nama variabel atau per karakter menggunakan indeks. Indeks dimulai dari 0, harus berupa bilangan bulat, dan dalam memori string disimpan secara berurutan seperti list karakter. Gambar 1.1 menunjukkan string berisi "banana" yang disimpan pada memory komputer.



Gambar 1.1: Bentuk string di dalam memory

C. Operator dan Metode String

a) OPERATOR in

Pada String kita dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain dengan menggunakan operator in. Hasil dari operator ini adalah True / False.

```
kalimat = "saya mau makan"
data = "saya"
print(data in kalimat) #True
print("mau" in kalimat) #True
print("dia" in kalimat) #False
```

Selain operator in, pada String juga dapat dilakukan perbandingan (comparison) yang juga menghasilkan True atau False.

```
if "saya" > "dia":
     print("Ya") #Ya
else:
     print("Tidak")

if  "dua" == "dua":
     print("Sama") #Sama
```

b) FUNGSI len

Cara untuk mengetahui berapa jumlah karakter dari sebuah string adalah dengan menggunakan operator len(). Untuk menampilkan huruf terakhir dari sebuah string kita harus menggunakan indeks string yang ke- len(-1), sebab indeks dimulai dari 0.

Contoh program Python adalah:

```
kalimat = "universitas kristen duta wacana yogyakarta"
print(len(kalimat)) #output 42

terakhir = kalimat[len(kalimat)-1]
print(terakhir) #output 'a'

#bisa juga menggunakan indeks -1
terakhir_versi2 = kalimat[-1]
print(terakhir_versi2) #output 'a'
#atau menggunakan indeks -2 untuk huruf terakhir kedua
terakhir2 = kalimat[-2]
print(terakhir2) #output 't'
```

c) TRAVERSING STRING

Untuk dapat menampilkan string dengan cara ditampilkan huruf demi huruf adalah dengan menggunakan loop yang dilakukan per huruf dengan 2 cara:

- Dilakukan dengan akses terhadap indeks:

```
kalimat = "indonesia jaya"
i = 0
while i < len(kalimat):
     print(kalimat[i],end='')
     i += 1
```

- Dilakukan tanpa akses terhadap indeks secara otomatis

```
kalimat = "indonesia jaya"
for kal in kalimat:
     print(kal,end='')
```

d) STRING SLICE

String slice adalah menampilkan substring pada sebuah string dengan menggunakan indeks dari awal tertentu sampai akhir-1 tertentu. Sintaksnya menggunakan [awal:akhir]. Bagian awal atau akhir boleh dikosongkan. Bagian awal dimulai dari 0.

```
kalimat = “suka kamu”
awal = 0
akhir = 4
print(kalimat[awal:akhir]) #suka
print(kalimat[5:len(kalimat)]) #kamu
print(kalimat[:3]) #suk
print(kalimat[3:]) #a kamu
print(kalimat[:]) #suka kamu
```

String merupakan data yang bersifat immutable! Immutable adalah bahwa data tersebut tidak bisa diubah saat program berjalan, hanya bisa diinisialisasi saja. Contoh:

```
kalimat = "satu"
kalimat[0] = "batu" #TypeError: 'str' object does not support item assignment
```

Agar bisa diubah, maka harus disimpan dalam variabel yang berbeda.

```
kalimat = "satu"
kalimat baru = kalimat[0] + "alah" #salah
```

Berikut adalah beberapa method String yang sering digunakan:

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Dan masih banyak lainnya. Ingat semua fungsi/method di atas yang mengembalikan string, mengembalikan string baru, tidak mengubah yang aslinya, karena string bersifat immutable. Daftar lengkap dapat dilihat di *https://docs.python.org/library/stdtypes.html#string-methods*

e) Operator * dan + pada String

Dalam Python, operator + dapat digunakan untuk menggabungkan dua string, sedangkan operator * digunakan untuk mengulang string sesuai jumlah perkaliannya. Perhatikan kode berikut:

```
kata1 = "aku"
kata2 = "suka"
kata3 = kata1 + " " + kata2
print(kata3) #hasil adalah penggabungan: aku suka
kata4 = "pale"
print(kata4 * 4) #hasil adalah palepalepalepale
kata4 = "pale "
print(kata4 * 2) #hasil adalah pale pale
```

D. Parsing String

Parsing string adalah proses menelusuri dan mengolah bagian-bagian string untuk mengambil atau mengubah informasi yang diinginkan. Contohnya, untuk mengambil tanggal dari kalimat, string dipecah menjadi beberapa token berdasarkan spasi, lalu dipilih token yang berisi tanggal (misalnya “17-08-1945”), dipecah lagi dengan tanda “-”, dan disusun ulang menjadi format yang diinginkan seperti 08/17/1945.

Berikut adalah contoh penyelesaiannya:

```
kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"
hasil = kalimat.split(" ")
for kal in hasil:
      if kal[0].isdigit():
            hasil2 = kal.split("-")
            print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

Hasil dari keluaran program adalah 08/17/1945

E. Pengantar Regex

Pengolahan string dengan cara biasa sering terasa sulit, sehingga digunakan teknik yang lebih efisien yaitu regular expression (regex). Regex adalah pola karakter untuk mencari, mengganti, atau menghapus bagian string tertentu, sehingga sangat membantu dalam parsing dibandingkan metode seperti split() dan find(). Meskipun powerful, regex cukup rumit dan tidak semua bahasa mendukungnya, namun Python menyediakan library regex melalui import re, dengan fungsi yang mudah digunakan seperti search().

Dengan menggunakn file mbox-short.txt, kita akan mencoba menampilkan semua string pada file tersebut yang mengandung pola "From: ".

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('From:', line):
count += 1
print(line)
print("Count: ",count)
```

Dari kode di atas kita dapat melihat bahwa re.search bisa saja diganti dengan menggunakan perintah find() pada string biasa. Pola pada contoh di atas belum menggunakan kemampuan regex yang seutuhnya.

From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
...
Jumlah: 27 *#[total dari seluruh kalimat dengan pola "From"]*

F. Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Libary Python

Sebelum menggunakan fungsi regex perlu diketahui terlebih dahulu meta character / special character dan kegunaannya pada pola regex seperti pada dua tabel di bawah.

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Pada Python terdapat beberapa penggunaan himpunan character dengan menggunakan simbol [], pada tabel di bawah.

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Pada Python terdapat 4 buah fungsi yang bisa dipakai untuk menggunakan Regex seperti pada tabel di bawah.

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link GitHub: https://github.com/fieliel-star/tugas-prakAlpro-minggu1-71251214.git

SOAL 1

```python
inpUser1 = input("masukan kata pertama: ")
inpUser2 = input("masukan kata kedua: ")

kata1 = ''.join([i.lower() for i in inpUser1 if i.isalpha()])
kata2 = ''.join([i.lower() for i in inpUser2 if i.isalpha()])

if sorted(kata1) == sorted(kata2):
    print("anagram")
else:
    print("bukan")
```

Penjelasan:

Program tersebut digunakan untuk mengecek apakah dua kata atau kalimat yang dimasukkan pengguna merupakan **anagram** atau bukan. Pertama, program meminta dua input dari pengguna, yaitu kata pertama dan kata kedua. Setelah itu, masing-masing input diproses dengan cara mengambil hanya karakter huruf saja (mengabaikan spasi, angka, atau simbol), lalu mengubah semua huruf menjadi huruf kecil agar perbandingan tidak terpengaruh oleh perbedaan kapitalisasi. Hasil dari proses ini disimpan dalam variabel kata1 dan kata2.

Selanjutnya, program membandingkan kedua kata tersebut dengan cara mengurutkan huruf-hurufnya menggunakan fungsi sorted(). Jika hasil pengurutan dari kedua kata sama, berarti kedua kata memiliki huruf yang sama dengan jumlah yang sama, sehingga dapat dikatakan sebagai anagram, dan program akan mencetak "anagram". Sebaliknya, jika hasilnya berbeda, maka program akan mencetak "bukan", yang berarti kedua kata tersebut bukan anagram.

Output:

```
masukan kata pertama: mata
masukan kata kedua: atma
anagram
PS D:\Semester_2\Praktikum
u09\soal1.py"
masukan kata pertama: mata
masukan kata kedua: roti
bukan
```

SOAL 2

```python
import re

kalimat = str(input("masukan kalimat: "))
cari =  str(input("masukan kata yang akan dicari: "))

kalimat = kalimat.lower()
cari = cari.lower()
kalimat = kalimat.replace(".", "")

x = re.split("\\s", kalimat)
jumlah = x.count(cari)

print(f"{cari} ada {jumlah} buah")
```

Penjelasan:

Program tersebut digunakan untuk menghitung berapa kali sebuah kata tertentu muncul dalam sebuah kalimat yang dimasukkan oleh pengguna. Pertama, program mengimpor modul re (regular expression) untuk membantu pemrosesan teks. Kemudian, pengguna diminta memasukkan sebuah kalimat dan kata yang ingin dicari. Setelah itu, kalimat dan kata tersebut diubah menjadi huruf kecil semua menggunakan .lower() agar pencarian tidak sensitif terhadap huruf besar/kecil. Program juga menghapus tanda titik dari kalimat dengan .replace(".", "") agar tidak mengganggu proses pemisahan kata.

Selanjutnya, kalimat dipecah menjadi daftar kata-kata menggunakan re.split("\s", kalimat), yang memisahkan berdasarkan spasi. Hasil pemisahan ini disimpan dalam variabel x. Program kemudian menghitung jumlah kemunculan kata yang dicari dalam daftar tersebut menggunakan x.count(cari) dan menyimpannya dalam variabel jumlah. Terakhir, program menampilkan hasilnya dengan format teks yang menyatakan berapa kali kata tersebut muncul dalam kalimat.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\
u09\soal2.py"
masukan kalimat: kamu adalah aku dan aku adalah kamu
masukan kata yang akan dicari: kamu
kamu ada 2 buah
```

SOAL 3

```python
import re

kalimat = str(input("masukan kalimat: ")).strip()

x = re.split(r"\s+", kalimat)
noSpace = ' '.join(x)

print(noSpace)
```

Penjelasan:

Program tersebut berfungsi untuk merapikan spasi dalam sebuah kalimat yang dimasukkan oleh pengguna. Pertama, program mengimpor modul re untuk menggunakan fitur regular expression. Kemudian, pengguna diminta memasukkan kalimat, dan fungsi .strip() digunakan untuk menghapus spasi berlebih di awal dan akhir kalimat. Setelah itu, kalimat dipecah menjadi daftar kata menggunakan re.split(r"\s+", kalimat), di mana pola \s+ berarti satu atau lebih spasi, sehingga pemisahan tetap benar meskipun terdapat spasi ganda.

Selanjutnya, daftar kata yang telah dipisahkan tersebut digabung kembali menjadi satu kalimat menggunakan ' '.join(x), sehingga setiap kata hanya dipisahkan oleh satu spasi saja. Hasilnya disimpan dalam variabel noSpace. Terakhir, program mencetak kalimat yang sudah dirapikan tersebut, yaitu tanpa spasi berlebih di antara kata-kata maupun di bagian awal dan akhir.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\7125
u09\soal3.py"
masukan kalimat: kamu  adalah aku dan  aku adalah  kamu
kamu adalah aku dan aku adalah kamu
```

SOAL 4

```python
kalimat = input("Masukkan kalimat: ")

kata_list = kalimat.split()

terpendek = kata_list[0]
terpanjang = kata_list[0]

for kata in kata_list:
    if len(kata) < len(terpendek):
        terpendek = kata
    if len(kata) > len(terpanjang):
        terpanjang = kata

print(f"terpendek: {terpendek}, terpanjang: {terpanjang}")
```

Penjelasan:

Program tersebut digunakan untuk mencari kata terpendek dan kata terpanjang dalam sebuah kalimat yang dimasukkan oleh pengguna. Pertama, program meminta input berupa kalimat, kemudian memecah kalimat tersebut menjadi daftar kata menggunakan fungsi .split(), yang secara otomatis memisahkan berdasarkan spasi. Setelah itu, elemen pertama dari daftar kata dijadikan sebagai nilai awal untuk variabel terpendek dan terpanjang, sehingga keduanya memiliki acuan awal untuk dibandingkan.

Selanjutnya, program melakukan perulangan untuk setiap kata dalam daftar tersebut. Pada setiap iterasi, panjang kata dibandingkan dengan panjang kata yang saat ini tersimpan sebagai terpendek dan terpanjang menggunakan fungsi len(). Jika ditemukan kata dengan panjang lebih kecil, maka variabel terpendek diperbarui, dan jika ditemukan kata dengan panjang lebih besar, maka variabel terpanjang diperbarui. Setelah seluruh kata diperiksa, program mencetak hasil akhir berupa kata terpendek dan terpanjang dalam kalimat tersebut.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\7
u09\soal4.py"
Masukkan kalimat: aku adalah kamu dan kamu adalah aku
terpendek: aku, terpanjang: adalah
```

SOAL 5

```python
import re
from datetime import datetime

def cari_dan_hitung_selisih(teks):
    list_tanggal = re.findall(r'\b\d{4}-\d{2}-\d{2}\b', teks)
    sekarang = datetime.now()

    for tgl_str in list_tanggal:
        tgl_obj = datetime.strptime(tgl_str, "%Y-%m-%d")
        selisih = (sekarang - tgl_obj).days
        print(f"{tgl_obj} selisih {selisih} hari")

teks = input("masukan teks: ")

cari_dan_hitung_selisih(teks)
```

Penjelasan:

Program tersebut berfungsi untuk mencari semua tanggal yang terdapat dalam sebuah teks, lalu menghitung selisih hari antara tanggal tersebut dengan waktu saat ini. Pada bagian awal, program mengimpor modul re untuk pencocokan pola teks (regular expression) dan datetime dari modul datetime untuk pengolahan tanggal dan waktu. Kemudian didefinisikan sebuah fungsi bernama cari_dan_hitung_selisih(teks) yang menerima input berupa string teks.

Di dalam fungsi ini, digunakan re.findall(r'\b\d{4}-\d{2}-\d{2}\b', teks) untuk mengekstrak semua pola tanggal dengan format YYYY-MM-DD (misalnya 2023-10-05) dari teks, dan hasilnya disimpan dalam list_tanggal. Setelah itu, variabel sekarang diisi dengan waktu saat ini menggunakan datetime.now().

Selanjutnya, program melakukan perulangan untuk setiap tanggal yang ditemukan dalam list_tanggal. Setiap tanggal dalam bentuk string (tgl_str) diubah menjadi objek datetime menggunakan datetime.strptime(tgl_str, "%Y-%m-%d") agar dapat dilakukan operasi perhitungan. Program kemudian menghitung selisih antara waktu sekarang dan tanggal tersebut dengan (sekarang - tgl_obj).days, yang menghasilkan jumlah hari (dalam bentuk bilangan bulat). Hasilnya ditampilkan menggunakan print, yang menunjukkan tanggal yang ditemukan beserta selisih harinya. Di bagian akhir, program meminta input teks dari pengguna, lalu memanggil fungsi cari_dan_hitung_selisih(teks) untuk memproses dan menampilkan hasilnya.

Output:

```
masukan teks: Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Patt
imura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-05-02).
1945-08-17 00:00:00 selisih 29464 hari
1785-11-11 00:00:00 selisih 87816 hari
1783-06-08 00:00:00 selisih 88703 hari
1889-05-02 00:00:00 selisih 50024 hari
```

SOAL 6

```python
import re
import random
import string

def generate_password(panjang=8):
    karakter = string.ascii_letters + string.digits
    return ''.join(random.choice(karakter) for _ in range(panjang))

def cari_email(teks):
    list_email = re.findall(r'\b[\w.-]+@[\w.-]+\.\w+\b', teks)

    for email in list_email:
        username = email.split("@")[0]
        password = generate_password(8)
        print(f"{email} username: {username} , password: {password}")

teks = input("Masukkan teks: ")
cari_email(teks)
```

Penjelasan:

Program tersebut berfungsi untuk mengekstrak alamat email dari sebuah teks sekaligus menghasilkan password acak untuk setiap email yang ditemukan. Pada bagian awal, program mengimpor tiga modul, yaitu re untuk pencocokan pola teks menggunakan regular expression, random untuk menghasilkan nilai acak, dan string untuk menyediakan kumpulan karakter seperti huruf dan angka. Selanjutnya didefinisikan fungsi generate_password(panjang=8) yang bertugas membuat password acak sepanjang 8 karakter (atau sesuai parameter). Di dalam fungsi ini, variabel karakter berisi gabungan huruf alfabet (string.ascii_letters) dan angka (string.digits). Password kemudian dibentuk menggunakan ''.join(random.choice(karakter) for _ in range(panjang)), yaitu memilih satu karakter secara acak berulang kali sebanyak panjang yang diinginkan, lalu menggabungkannya menjadi sebuah string.

Berikutnya terdapat fungsi cari_email(teks) yang digunakan untuk mencari semua email dalam teks yang diberikan. Program menggunakan re.findall(r'\b[\w.-]+@[\w.-]+\.\w+\b', teks) untuk menemukan pola email, di mana pola tersebut mencocokkan kombinasi karakter huruf, angka, titik, atau tanda hubung sebelum dan sesudah simbol @, serta diakhiri dengan domain seperti .com atau lainnya. Semua email yang cocok disimpan dalam list_email. Program

kemudian melakukan perulangan untuk setiap email yang ditemukan. Pada setiap iterasi, bagian username dari email diambil dengan email.split("@")[0], yaitu bagian sebelum tanda @. Setelah itu, fungsi generate_password(8) dipanggil untuk membuat password acak sepanjang 8 karakter. Hasilnya kemudian dicetak dengan format yang menampilkan email, username, dan password yang dihasilkan. Di bagian akhir, program meminta input teks dari pengguna melalui input("Masukkan teks: "), lalu memanggil fungsi cari_email(teks) untuk memproses teks tersebut, sehingga setiap email yang ditemukan akan ditampilkan bersama username dan password acaknya.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel
u09\soal6.py"
Masukkan teks: anton@mail.com
anton@mail.com username: anton , password: LgoqIBdC
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel
u09\soal6.py"
Masukkan teks: 71251214@students.ukdw.ac.id
71251214@students.ukdw.ac.id username: 71251214 , password: rhMiRklR
```